Four Linux Systems Projects: ELF Forensics, System Call Tracing, a CPython Extension and a Signal Service

Louis | African Leadership University | Low Level Programming

GitHub Repo

Abstract

This report covers four projects on how Linux programs behave below the source code. Project 1 compares two ELF executables using static analysis only. Project 2 traces a backup program with strace and compares a successful run with a failed one. Project 3 builds a CPython C extension for statistics and measures it against pure Python. Project 4 builds a service that is controlled with Unix signals. Each project was compiled with GCC on Ubuntu Linux (x86-64, Python 3.12.3) and tested with scripted commands. The report explains the design choices, the evidence collected, the conclusions that evidence supports, and where each design falls short.

Introduction and Approach

The four tasks share one question: what does a program really do once it is compiled and running? Each one answers it with a different tool, so I used the same method throughout. Build the program from source I control, collect evidence with a tool, and only then state a conclusion. Where the evidence only suggests something, the report says so.

Every C program is split into small modules, each with a header that states what the module promises and a source file that does the work. Modules return status codes and do not print, and the main file turns those results into messages. I wrote and tested one module at a time. The Makefiles use `-Wall -Wextra`, and every build in this report finished without warnings. Project 1 and Project 2 use `-std=c11` and `-std=gnu11`. Strict C11 hides POSIX declarations such as `fork` and `mkdir`, so Project 2 needed the GNU variant.

Overview of the Four Projects

Project	Subject	Main tool	Core evidence	Main result
1. ELF forensics	`data_sync` and `data_sync_v2`, built from 3 and 6 modules	file, readelf, nm, objdump	26 saved output files	Same structure, 13 more imported functions in v2
2. System call tracing	`backup_tool` with a forked copy process	strace	3 trace files	A missing source shows as `openat` returning ENOENT
3. CPython extension	`faststats` in C against `pystats` in Python	perf_counter, setuptools	76 equivalence	Identical results, 7.3x to 46.9x faster

Project	Subject	Main tool	Core evidence	Main result
			checks, 15 timings	
4. Signal service	`server_controller`	kill, sigaction, pselect	Timed demo log	All four signals handled, shutdown paths differ

The sections below follow the same order for each project: what was built and why, the evidence, how failures are handled, and the limits.

Techniques Used Across the Projects

Several ideas appear in more than one project. Resources are released on every exit path: Project 2 closes both file descriptors even when a read fails, and Project 3 frees its C array on every error return. Errors are returned and read from `errno`, then saved before a later call such as `close()` can overwrite them. Output is kept apart from logic, so the same function can be traced, tested or reused. Finally, library calls do not map one to one onto system calls. In the evidence, `fork` became `clone`, `waitpid` became `wait4`, `stat` became `newfstatat`, `printf` became `puts` and `fprintf` became `fwrite`. Reading the evidence correctly depends on knowing this.

Project 1: ELF Binary Forensics

What was built

Two programs simulate a file synchronization utility. `data_sync` is made of a scanner module, a compare module and a main file. It reads two directories with `opendir` and `readdir`, sorts the names, and walks both sorted lists once to find names missing from the destination. `data_sync_v2` links the same two modules plus a logger, a metadata module that calls `stat`, and a checksum module that applies the 32 bit FNV-1a hash. Sharing the first two modules means the differences between the binaries come from the added code and not from unrelated changes.

Method

The assignment forbids running unknown binaries, so the investigation used `file`, `readelf`, `nm` and `objdump` only, and never `ldd`. One script, `analyze.sh`, runs every command on both binaries and writes 26 output files, so each finding can be traced to a file.

Evidence

Property	data_sync	data_sync_v2
Class, order, machine	ELF64, little endian, x86-64	same
Type	DYN (position independent executable)	same
Entry point	0x11e0	0x1380

Property	data_sync	data_sync_v2
Program headers, sections	13 and 31	13 and 31
`.text` size	0x726	0x106e
Symbol table entries	57	83
Imports through the PLT	13	26
Dependency	libc.so.6 only	libc.so.6 only
Debug sections, stripped	none, not stripped	same

The structure is identical, and the capability differs. The 13 extra imports in v2 match its new modules: time functions for the logger, `stat` for metadata, and `fopen`, `fread` and `ferror` for the checksum. The relocation count agrees with the import count, which is a useful cross-check: `.rela.plt` holds 13 entries in v1 and 26 in v2.

Three observations are worth recording. First, the entry point equals the start of `.text` in both binaries, so its difference follows from a larger PLT in v2 and says nothing about behaviour. Second, `main` appears in `.symtab` but not in `nm -D`, because the dynamic symbol table holds only what the loader needs. A stripped build would remove `main` from view while the imports stayed visible. Third, `fwrite` and `puts` appear in the import list although the source calls `fprintf` and `printf`. GCC replaces a call that has no format arguments with the cheaper function, so the imports do not mirror the source line for line.

Sections and segments

Sections are the linker's view and segments are the loader's view. In both binaries four LOAD segments (read, read and execute, read, read and write) group many sections by permission. `.symtab`, `.strtab` and `.comment` belong to no segment because they are never loaded, which is why stripping does not change how a program runs.

A finding about the method

The symbol table showed `compare_names` as global (`T`) when the source declared it `static` (`t`). An earlier edit had removed the keyword by mistake. The binary exposed the difference, I corrected the source, rebuilt, and regenerated all 26 files so the evidence matches the final code.

Limits

Imports show that a function could be called, not that it is. Static inspection cannot show which files are touched or whether a code path is reachable. The binaries were built from my own source, so the conclusions describe a controlled case. A real unknown file would also need dynamic analysis in an isolated environment.

Project 2: System Call Forensics

What was built

`backup_tool` creates a backup directory, writes a numbered log, and copies a file. A forked child does the copy and the parent waits. The code uses `open`, `read`, `write`, `mkdir`, `fork` and `waitpid` directly instead of `fopen` and `fprintf`, because buffered I/O would hide which source line caused which system call. The copy loop handles short writes and `EINTR`. The log uses `O_APPEND` because the child inherits the parent's descriptor and both write to it.

Evidence

	Success run	Failed run
Source open	`openat(..., "testdata/report.txt", O_RDONLY) = 4`	`openat(..., "testdata/missing.txt", O_RDONLY) = -1 ENOENT`
Destination open	`openat(..., "backup_out/report.txt", ...) = 5`	not attempted
Data	`read` 15 bytes, `write` 15 bytes, `read` returns 0	none
Child exit	`exit_group(0)`	`exit_group(1)`
`wait4` result	`WEXITSTATUS(s) == 0`	`WEXITSTATUS(s) == 1`
Parent exit	`exit_group(0)`	`exit_group(1)`

The trace shows the cause of the failure directly: the kernel refused the first `openat` with ENOENT and nothing after it touched the destination. The program also handles it correctly. The child checks the return value, logs the `errno` text, closes its descriptor and exits with 1, and the parent decodes the status and exits with 1. Running the tool without strace printed exit status 1, which agrees with the trace.

Process calls show the cooperation. The parent calls `clone`, receives the child PID and blocks in `wait4`. The child does all the copy work and ends with `exit_group`. The parent then resumes with the child's status. No `execve` follows the `clone`, because the child runs the same program image. A third trace of a rerun on an existing directory shows `mkdir` returning EEXIST, followed by `newfstatat` with `S_IFDIR`, which is the path through `ensure_dir()` that accepts a directory that is already there.

Program calls and runtime calls

The start of each trace holds calls that the program did not make: `execve` from the shell, then `access` of `ld.so.preload`, `openat` of `ld.so.cache` and `libc.so.6`, and `fstat` calls from the dynamic loader and stdio. Only calls after the loader finishes belong to the program. The ENOENT on `ld.so.preload` is normal and is not an error in the program.

Unexpected behaviour and improvement

In the log, sequence numbers 3 and 4 appear twice. After `fork()` each process holds its own copy of the counter, so both continue from the same value. The failure run exposed a design weakness: the tool printed "Backup directory ... OK" and created the directory before it found out that the source was missing. Checking the source with `stat()` before `mkdir` and `fork` would fail early and leave nothing behind.

Limits

The copy writes straight into the destination, so a failure in the middle would leave a partial file. By reading the code, I expect that, but I did not test it. Only one failure type was traced. A permission failure would show EACCES instead of ENOENT, and I did not run that experiment.

Project 3: CPython C Extension

What was built

`pystats.py` implements mean, population variance, minimum and maximum, a count of values outside two bounds, and min-max normalization in pure Python. `faststats.c` implements the same five operations through the CPython C API. A helper, `to_doubles()`, validates a sequence and converts it into a C array of doubles in one pass. Every function then works on plain doubles, frees the array on every exit path, and returns a Python object or sets an exception.

Both versions follow one set of rules so that they are comparable. Empty input raises `ValueError`, except that `count_outside` returns 0. Booleans and non-numeric items raise `TypeError`, and bool is rejected explicitly because it is a subtype of int. A constant dataset normalizes to zeros instead of dividing by zero.

How Python reaches the C code

Each exposed function has the form `PyObject *fn(PyObject *self, PyObject *args)`. `PyArg_ParseTuple` unpacks the arguments, and the function returns a new object, or returns NULL after setting an exception. A `PyMethodDef` table lists the functions, a `PyModuleDef` describes the module, and `PyInit_faststats` is the entry point that `import` looks for. `setup.py` drives two gcc commands, which are visible in the build log: one compiles with `-I/usr/include/python3.12` and `-fPIC`, and one links with `-shared` into `faststats.cpython-312-x86_64-linux-gnu.so`. The file name carries the Python version and platform.

Correctness

`test_equivalence.py` runs 76 checks, and all passed. They cover small datasets, single and constant values, negatives, mixed int and float, tuples, seeded random data at 1,000, 100,000 and 1,000,000 values, empty input, seven kinds of invalid input, bad bounds and return types. The comparison is exact equality with no tolerance. Both versions add in the same order and the build has no `-ffast-math`, so a tolerance would only hide a real difference. NaN and integers above 2^53 are deliberately not covered, because comparison order changes results for NaN and C compares after conversion to double.

Performance

Average speedup is Python time divided by C time, over 50, 10 and 3 repeats for the three sizes.

Operation	1,000	100,000	1,000,000
mean	26.3x	26.9x	20.9x
variance	44.2x	46.9x	42.7x
minmax	24.3x	20.1x	20.9x
count_outside	24.3x	24.9x	22.4x
normalize	13.8x	10.0x	7.3x

Every operation was faster at every size. The simple operations cluster between 20x and 27x, because the C version still has to read every Python object once to convert it. `variance` gains most, but part of that comes from the baseline: the Python version validates the data twice and loops twice, so it is not a clean comparison of loops. `normalize` gains least, and its speedup falls from 13.8x to 7.3x, because the result must be a list of Python floats, and creating a million float objects costs the same in both versions. Speedups held steady from 1,000 to 100,000 values and dipped slightly for some operations at 1,000,000. Cache pressure and memory allocation are plausible causes, but the benchmark did not isolate them, so they are hypotheses.

Trade-offs and limits

The C version is much faster, but a mistake in it can leak memory or crash the whole interpreter, where Python would raise an exception. The module must be rebuilt for each Python version and platform. The C functions copy the data into a second array, and very large integers raise `OverflowError`. The 1,000,000 value case used only 3 repeats, so those averages are the least reliable. All timings belong to this machine. Because the input is a Python list, the speedup includes the cost of conversion, and an array type would avoid it, but that was outside the assignment.

Project 4: Signal Driven Service Controller

What was built

`server_controller.c` prints a status line every five seconds and keeps a state of NORMAL or MAINTENANCE. SIGUSR1 prints a status report with the state and PID. SIGUSR2 toggles the state. SIGINT starts a graceful shutdown, and SIGTERM ends the service at once.

How signals are handled safely

Only a few functions are safe inside a signal handler, and `printf` is not one of them. The handler therefore does one thing: it sets a `volatile sig_atomic_t` flag for the signal it received. All printing, state changes and cleanup happen in the main loop. The program registers the handler with `sigaction()` instead of `signal()`, because POSIX defines its behaviour and it does not vary between systems.

The wait is the part that needed the most thought. With `sleep(5)`, a signal that arrives just after the flag check but before the sleep starts would be noticed up to five seconds late. The program blocks the four signals while it works and waits with `pselect()` and a signal mask. `pselect()` unblocks them atomically only during the wait, so a signal that arrives earlier stays pending and ends the next wait at once. A monotonic deadline keeps the tick steady, so an interrupted wait resumes with the time that remains. Flags also merge repeated signals, so two SIGUSR2 signals that arrive before the loop runs cause one toggle.

Evidence

A script, `demo.sh`, starts the service, sends each signal with `kill` at timed intervals, and writes the output to `demo_log.txt`.

Signal	Observed response	Exit status
SIGUSR1	Status report with the state and PID 22556, each time	keeps running
SIGUSR2	NORMAL to MAINTENANCE and back, and the next tick message changed with it	keeps running
SIGINT	Shutdown and cleanup messages, PID file removed	0
SIGTERM	Emergency message, PID file left behind	1

Ticks stayed about five seconds apart even though signals arrived between them. The PID file makes the difference between the two shutdowns visible. SIGINT runs cleanup and removes it, and SIGTERM skips cleanup and leaves a stale file. In many real services SIGTERM is the polite shutdown request, so this behaviour follows the assignment's definition and not common practice.

Limits

SIGKILL and SIGSTOP cannot be caught, so they bypass all of this. The PID file is written without checking for a running second instance. The service has no real monitoring work, only a message. The demo was run by a script and not typed by hand, which the log header shows.

Testing Methodology and Results

Each project was tested by running scripted commands and reading the output, so each scenario can be repeated. The evidence differs by project because the tools differ.

Project	What was run	Result
1	`analyze.sh` on both binaries, 26 output files, rerun after the `static` fix	Findings consistent across tools, such as 13 and 26 relocations agreeing with the import counts
2	Successful run, failed run, rerun on an existing directory, all under strace	Each outcome matched the trace

Project	What was run	Result
3	76 equivalence checks, then 15 timed measurements	All passed, all faster
4	Scripted run of all four signals, with exit statuses and a PID file check	Each response matched the specification

There are gaps. Project 2 tested one failure type, and no run injected an allocation or `fork` failure, so those paths were checked by reading the code. I did not run the programs under AddressSanitizer or Valgrind. Everything ran on one machine (Ubuntu, GCC, Python 3.12.3). The tests are scripts that I rerun by hand and not an automated suite. Timing results depend on the machine, and the 1,000,000 value timings used only 3 repeats.

Limitations

Beyond the limits in each section, two apply to the whole work. The conclusions come from programs I wrote, so they show the method works on a controlled case and not that it would catch every real problem. The evidence files are specific to one run, and PIDs, timestamps and addresses change when anything is rebuilt, so the quoted numbers belong to the files submitted with this report.

Practical Applications

Reading ELF structure, imports and symbols is the first step in triaging an unknown executable. It shows what a binary could do before any risk is taken. System call tracing is the standard way to diagnose a program that fails in production, and the permission and missing file cases in Project 2 are everyday examples. C extensions are used where Python code is too slow and the work fits a small, well defined function. Signal handling underlies how service managers stop, reload and report on daemons.

Conclusion and Next Steps

All four projects met their tasks, and the evidence supported the conclusions drawn. The most useful lesson was that tool output needs interpreting. An import is not an action, a library call is not one system call, and a speedup is not a pure measure of C against Python when the baseline repeats work. Several corrections came from checking the evidence against the source, such as the `static` keyword in Project 1.

These changes would improve the work most:

1. Put each project's checks into a script that compares output automatically, so a change cannot break something unnoticed.
2. In Project 2, check the source file before creating the directory, write the copy to a temporary name and rename it when it is complete, and add the PID to log entries.
3. In Project 2, run a permission failure experiment as well, to show EACCES in a trace.
4. In Project 3, raise the repeat count at 1,000,000 values and measure the cache hypothesis with a profiling tool.
5. In Project 4, refuse to start if a live process already owns the PID file.

References

Kerrisk, M. (2010). The Linux programming interface. No Starch Press.

Python Software Foundation. (2024). Extending and embedding the Python interpreter, and Python/C API reference manual. https://docs.python.org/3.12/

The Open Group. (2018). The Open Group base specifications, issue 7 (IEEE Std 1003.1-2017): sigaction, pselect, fork, waitpid. https://pubs.opengroup.org/onlinepubs/9699919799/

TIS Committee. (1995). Tool interface standard (TIS) executable and linking format (ELF) specification, version 1.2.

Appendix: Build and Run

Run each command from inside the project folder.

Project	Folder	Build	Run or analyze
1	project1	`make`	`sh analyze.sh`
2	project2	`make`	`strace -f -tt -o traceA_success.txt ./backup_tool testdata/report.txt backup_out`
3	project3	`python3 setup.py build_ext --inplace`	`python3 test_equivalence.py`, then `python3 benchmark.py`
4	project4	`gcc -Wall -Wextra -o server_controller server_controller.c`	`sh demo.sh`

Full instructions are in README.md.